

getarp.net — Defence Intelligence PoC: High-Level Design

Author: Security Architecture (15y) + 2 senior engineers **Status:** PoC / v0.1 **Target:** Single VM, 8 vCPU / 64 GB RAM, domain `getarp.net`

1. Objective

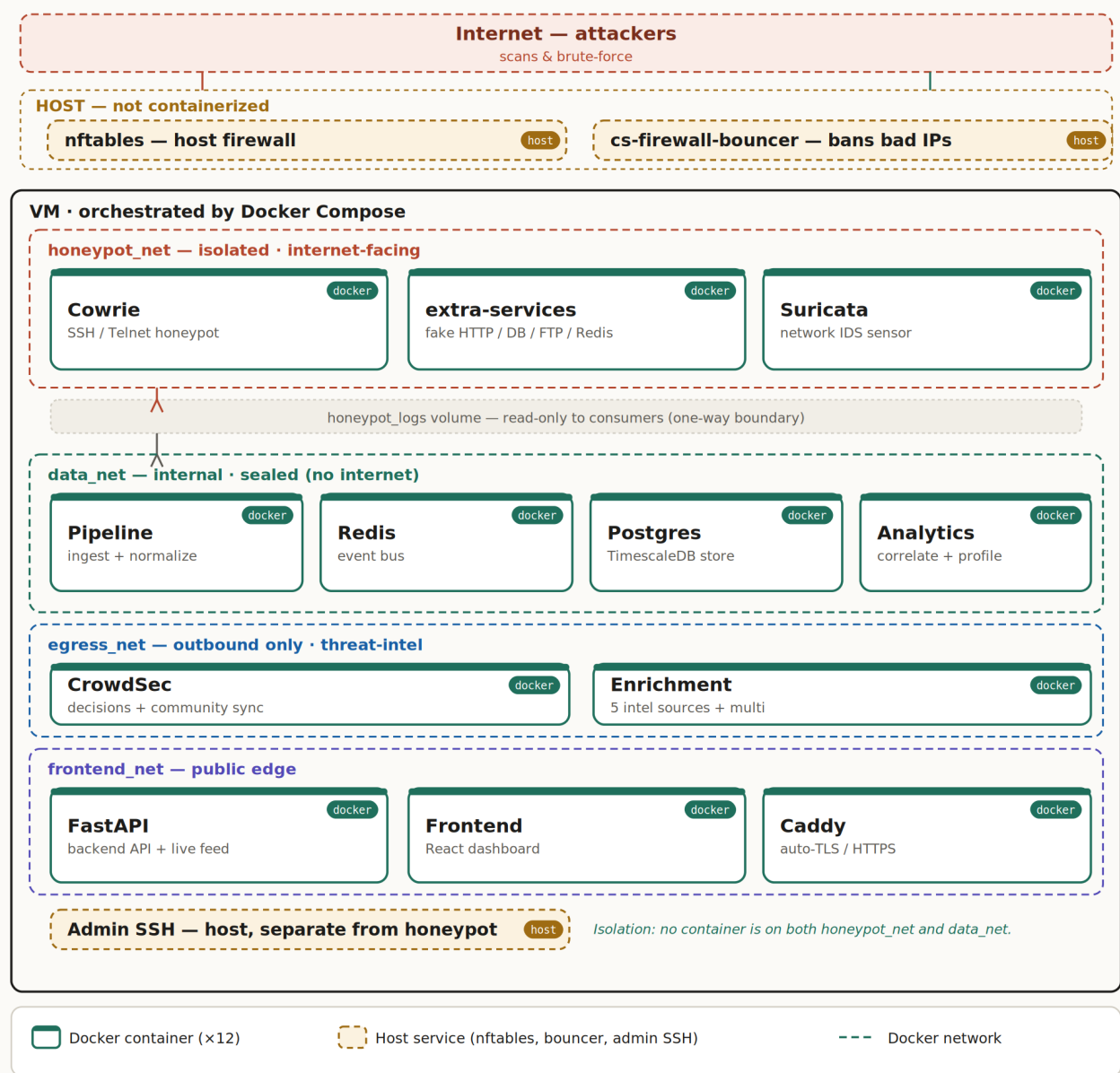
Stand up an internet-exposed deception sensor (Cowrie + multi-service emulator), capture real attacker traffic, run an IDS over it, enrich every observed IP with external threat intelligence (CrowdSec, swappable), correlate scans vs. attacks, profile attacker behaviour, and surface all of it through a public dashboard and an authenticated admin backend — all on one VM, built to be modular so components (intel provider, correlation engine, an AI module) can be swapped or added later.

2. The one principle that drives the whole design

A honeypot is built to be compromised. Treat the sensor as hostile.

Everything else follows from this. The internet-facing deception layer is placed in an isolated network segment that **cannot reach** the data store, the API, or the admin plane. Telemetry leaves the honeypot the only safe way it can: as **append-only log files on a read-only-to-consumers volume**, pulled by a collector. There is no network path from a popped honeypot into your intelligence database.

3. Logical architecture



Network segmentation is enforced with three Docker networks. The honeypot network is `internal: false` only on the published attack ports; it has `no member` that also sits on `data_net`. The pipeline never connects to the honeypot — it only reads the shared log volume.

4. Component responsibilities

Layer	Component	Tech	Role
Deception	Cowrie	cowrie/cowrie	Medium-interaction SSH/Telnet, fake FS, fake creds
Deception	Extra-Services	custom asyncio	HTTP/MySQL("DB")/FTP/Redis banners, logs auth attempts
Detection	Suricata	jasonish/suricata	

Layer	Component	Tech	Role
			IDS on the public NIC, ET Open + custom rules → <code>eve.json</code>
Intel/ Enforcement	CrowdSec	<code>crowdsecurity/crowdsec</code>	Parses sensor logs, makes decisions, CTI enrichment, community blocklist
Enforcement	cs-firewall-bouncer	bouncer	Applies CrowdSec decisions to host nftables
Bus/Cache	Redis	<code>redis:7</code>	Event stream + pub/sub for live status
Storage	PostgreSQL + TimescaleDB	<code>timescale/timescaledb</code>	Events, sessions, IPs, enrichment, correlations, reports, users
Ingest	Pipeline	Python	Tails sensor logs → normalize → Redis stream + Postgres
Intel	Enrichment	Python	Per-IP enrichment behind a common interface — 5 providers (crowdsec, abuseipdb, greynoise, virustotal, ciscotalos) + multi
Analytics	Analytics	Python	Pluggable correlation (scan/attack), behavioral profiling, 5-min status, reports
API	Backend	FastAPI	Auth, settings, data endpoints, WebSocket live feed
UI	Frontend	React + Vite	Dashboard, attacker map, correlation/behavior views, reports, settings
Edge	Caddy	<code>caddy:2</code>	Reverse proxy + automatic HTTPS for <code>getarp.net</code>

5. Canonical event schema (the contract that makes it modular)

Every sensor is normalized to one schema before anything downstream touches it. Swap a sensor or a provider, and nothing else changes as long as it emits/consumes this:

```
{
  "event_id": "uuid",
  "ts": "RFC3339",
  "sensor": "cowrie|suricata|extra",
  "service": "ssh|telnet|http|mysql|ftp|redis|...",
  "event_type": "connect|login_attempt|login_success|command|alert|scan|file",
  "src_ip": "x.x.x.x",
  "src_port": 0,
  "dst_port": 0,
  "username": null,
  "password": null,
  "command": null,
  "signature": null,
  "severity": 0,
  "session": "cowrie-session-id",
```

```
"raw": { }  
}
```

`event_type` + `service` are the only fields most downstream modules need. The `raw` blob preserves the original record for forensics and for future modules (e.g. an AI module that wants the full command transcript).

6. Data flow

1. Attacker hits Cowrie / Extra-Services / triggers a Suricata rule.
2. Sensors append JSON to `/data/logs/*.json`.
3. **Pipeline** tails those files, normalizes to the canonical schema, `XADD`s to the Redis stream `events`, and bulk-inserts into the `events` hypertable. New IPs are pushed to the `enrich:queue` stream.
4. **Enrichment** consumes `enrich:queue`, calls the active provider — one of five (crowdsec, abuseipdb, greynoise, virustotal, ciscotalos) or the multi-aggregator (CrowdSec CTI by default), upserts `ip_enrichment`.
5. **Analytics** consumes the `events` stream: - `ScanDetector` flags an IP touching $\geq N$ distinct ports in a window $\rightarrow$ `scan_events`. - `AttackDetector` flags brute force / post-auth commands / IDS exploit sigs $\rightarrow$ `attack_events`. - `BehavioralProfiler` maintains per-IP profiles + a threat score, clusters tactics. - Every 5 min, `StatusAggregator` writes a `status_snapshots` row and publishes it on Redis pub/sub `status:live`. - On schedule, `ReportBuilder` produces period reports.
6. **CrowdSec** independently parses the same logs, issues decisions; the firewall bouncer drops offenders at nftables. (Detection for analytics is decoupled from enforcement so a bad rule can't blind your dashboard.)
7. **API** reads Postgres for REST, subscribes to `status:live` and pushes to the UI over WebSocket. **Frontend** renders; also polls `/api/status` every 5 min as a fallback.

7. Modularity / extension points (explicitly requested)

- **Intel providers — five built in, easily extended:** the system ships with five providers behind a common `EnrichmentProvider` interface, plus a `multi` aggregator:

Provider	Key required	Notes
<code>crowdsec</code>	Optional (CTI key)	Default single-provider mode; falls back to local LAPI decisions
<code>abuseipdb</code>	Yes	Free tier: 1,000 checks/day
<code>greynoise</code>	Optional	Community API works without a key, limited results
<code>virustotal</code>	Yes	Free tier: 500 lookups/day
<code>ciscotalos</code>	No	Talos reputation feed; no key needed
<code>multi</code>	—	Recommended — queries all configured providers in parallel and merges verdicts

Add a sixth by subclassing `EnrichmentProvider.enrich(ip)` in `enrichment/providers.py`, decorating with `@register`, and flipping `ENRICHMENT_PROVIDER` in `.env`. The `multi` merge strategy is worst-of reputation, max confidence, any-attacker flag, union of categories (source-prefixed), with each provider's sub-result

preserved in `raw` for forensics; a provider with a missing key is silently skipped rather than failing the lookup. - **Add a correlation/behavioral module:** subclass `Detector` in `analytics/correlation/` (or a profiler in `analytics/behavioral/`), drop it in the registry. Engine auto-loads enabled detectors from config. - **Add an AI module later:** the behavioral profiler exposes a `score(profile, events)` hook and the canonical `raw` transcripts are persisted, so an ML/LLM scorer can be added as its own container that consumes the `events` stream and writes back to `attack_events.ai_score` — no schema migration needed. - **Scale out:** Redis Streams + consumer groups means you can run N analytics workers, or lift the whole `data_net` tier to a second VM and point the pipeline's Redis at it. Postgres → managed PG; Cowrie → multiple sensors writing to the same bus.

8. Resource budget (8 vCPU / 64 GB)

Service	CPU (cores)	RAM	Notes
Suricata	1.5	2–4 GB	Honeypot traffic is low-volume; cap AF_PACKET threads
Cowrie + Extra	0.5	1 GB	Mostly idle, spikes on scans
CrowdSec + bouncer	0.5	1 GB	
Postgres/TimescaleDB	2	8–16 GB	<code>shared_buffers</code> 8 GB, room for retention
Redis	0.25	1 GB	Streams trimmed
Pipeline / Enrichment / Analytics	1.5	3 GB	
API + Frontend + Caddy	0.5	1 GB	
Reserved for future AI module	~1.25	~30 GB	Headroom is deliberate

Plenty of slack — the box is comfortably oversized for PoC traffic, which is the right call so the future AI/correlation work has somewhere to land.

9. Security model (beyond the isolation principle)

- **Admin plane is not for the public.** The login/settings backend should be bound behind an allow-list or VPN. The Caddyfile ships with a commented `@admin` matcher + IP allow-list; turn it on. Public users get read-only dashboard routes only.
- **Keep admin SSH separate from the honeypot.**
- **Least privilege containers:** honeypot containers run non-root where possible, read-only rootfs, `cap_drop ALL`, no Docker socket, resource limits to contain a fork-bomb attempt.
- **Secrets** via `.env` / Docker secrets, never in images. JWT secret + DB creds + provider API keys are all env-injected.
- **Retention & PII:** captured payloads can contain attacker (and occasionally third party) data. Set retention windows in `db/init.sql` Timescale policies; document lawful basis since this is "Defence Intelligence."

10. Live status & reporting

- **Live (≤5 min):** `StatusAggregator` runs on a 300 s tick, snapshots active-attackers / new-IPs / events-per-min / top targets / threat-level, publishes on `status:live`. UI updates over WebSocket instantly and via 5-min poll fallback.
- **Reports:** `ReportBuilder` renders period summaries (top attackers, scan vs. attack breakdown, new TTPs, geo/ASN distribution) to HTML/JSON, downloadable from the UI and generated on a daily schedule.

11. What is PoC-grade and would need hardening for production

Honest scope: this scaffold gives you a **working, modular system**. Before it's "production Defence Intelligence" you'd want: HA Postgres + backups, proper secrets manager, mTLS between services, signed/audited admin actions, SOAR/ticketing integration, formal data-retention & legal review, and load-testing Suricata against real line-rate traffic. Those are called out at each component's README.